

VITALS-Edge

Telemetry Energy Field + Residual Digital Twin Dashboard

Development Blueprint and Run Flow

VITALS-Edge Telemetry Energy Field + Residual Twin Architecture

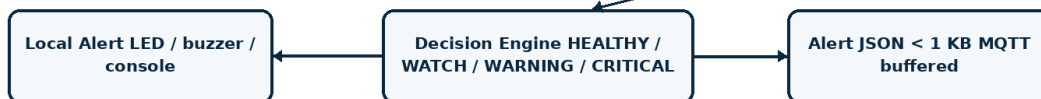
1. Telemetry Sources



2. Edge Intelligence



3. Thin Cloud Store



4. Dashboard Experience



Key claim: raw high-rate telemetry stays at the edge; the cloud receives compact validated events and dashboard-ready su

A buildable MVP for a telemetry-first dashboard that converts raw engine signals into a visual energy field, residual digital twin drift, ML health state, RUL estimate, and actionable maintenance alerts.

Team Algorithm404 | AI & Data Science | Hackathon Development Document

Quick Navigation

Section	What it contains
1. Final Idea	Dashboard-centred VITALS-Edge concept
2. Core Flow	Telemetry -> residual twin -> energy field -> ML -> dashboard
3. Dashboard Design	Screens, widgets, and judge-visible story
4. Technical Architecture	Modules, schemas, storage, APIs
5. Development Runbook	Build order, commands, tests, demo flow
6. Model Plan	Twin predictor, anomaly model, RUL model, classifier
7. Acceptance Gates	KPIs, risks, and final demo checklist

1. Final Project Idea

Project name: VITALS-Edge Telemetry Energy Field Dashboard

One-line definition: A telemetry-based vehicle/engine health dashboard that transforms live sensor data into an energy-field heatmap, compares actual behavior with a residual digital twin, runs edge ML for anomaly + RUL prediction, and gives a clear maintenance decision in real time.

This is not just a normal IoT dashboard. The dashboard becomes the main intelligence window: it shows where the machine is becoming abnormal, why the ML model thinks it is abnormal, how far the actual system has drifted from the healthy digital twin, and how much useful life remains.

1.1 Why this version is strong

- It is telemetry-first: the project has visible live data, not only a trained model in a notebook.
- It is edge-first: the heavy computation and alert decision can run on a laptop/Jetson even when the network is disconnected.
- The energy field gives a unique visual identity. Judges can immediately see component stress instead of only numbers.
- The residual digital twin makes the ML explainable: actual telemetry is compared against expected healthy behavior.
- The dashboard is demoable without a real engine because dataset replay can behave like live telemetry.

1.2 MVP Scope Lock

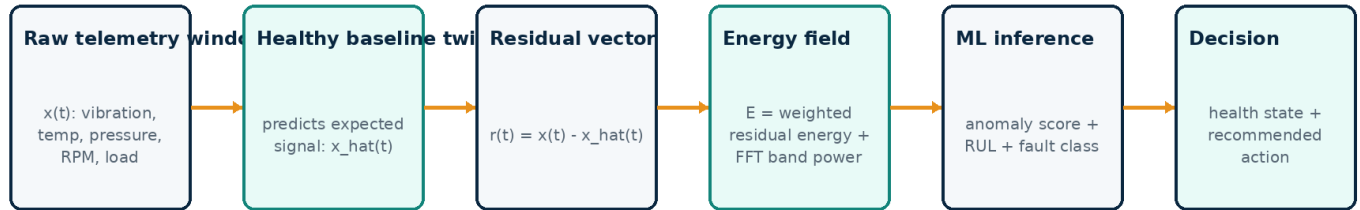
Keep in MVP	Do not include in MVP
Vibration, temperature, pressure, RPM/load telemetry	Full 3D Unity digital twin
Energy-field heatmap and health index	AR maintenance interface
Residual digital twin as time-series predictor	Full production fleet system
Anomaly detection + RUL estimate	Computer vision inspection unless time remains
Streamlit dashboard + FastAPI edge service	Complex cloud AI pipeline

2. Core System Flow

The final flow must be very simple to explain and very strong technically:

```
[Telemetry input]
-> [Shared preprocessing]
-> [Residual Digital Twin: expected vs actual]
-> [Energy Field Engine: stress/anomaly energy per component]
-> [ML Models: anomaly score + RUL + fault class]
-> [Decision Engine: HEALTHY/WATCH/WARNING/CRITICAL]
-> [Local alert + dashboard + compact cloud packet]
```

Residual Twin + Energy Field + ML Model Flow



Core equations used by the prototype

- 1) Expected telemetry: $x_hat_t = \text{TwinModel}(x_{\{t-W:t\}}, \text{operating_context})$
- 2) Residual: $r_t = x_t - x_hat_t$
- 3) Residual energy: $RE_t = r_t^T W r_t$
- 4) Energy field cell: $E_component = \text{RMS}(r_window)^2 + \alpha * \text{FFT_band_power} + \beta * \text{temp_gradient}$
- 5) Health index: $HI = 100 * \exp(-\lambda * \text{total_energy})$

2.1 Data Sources

Source	Signals	Use in MVP
Dataset replay	NASA C-MAPSS cycles, CWRU vibration, AI4I tabular machine records	Acts like live telemetry when no engine is available
Vehicle/CAN later	RPM, engine load, coolant temp, MAP, speed	Context and normalization for real deployment
Sensor rig later	Accelerometer, temperature, pressure, current	Physical proof-of-concept if hardware is available

2.2 The SensorWindow Payload

Every input becomes a validated SensorWindow. The same object is used for dataset replay and real sensors, so the demo path and field path are identical.

```
SensorWindow = {
  "asset_id": "ENG-01",
  "timestamp": "2026-07-03T10:30:00",
  "sample_rate_hz": 12000,
  "window_size": 1024,
  "channels": {
    "vibration_x": [...],
    "temperature": [...],
    "pressure": [...],
    "rpm": [...],
    "load": [...]
  }
}
```

3. Energy Field Engine

The energy field is the signature idea. It converts telemetry into a component-level stress map. It does not need to claim physical electromagnetic energy. In this project, “energy” means computed abnormal energy from residuals, vibration bands, temperature gradients, and pressure/load deviations.

3.1 Energy Field Formula

```
residual r_t = actual_signal x_t - expected_signal x_hat_t

residual_energy RE_t = r_t^T W r_t

component_energy = RMS(residual_window)^2
+ alpha * FFT_band_power
+ beta * temperature_gradient
+ gamma * pressure_deviation

health_index = 100 * exp(-lambda * total_energy)
```

3.2 What the Energy Field Shows

Dashboard field	Meaning	Example
Bearing energy	Abnormal vibration residual near bearing frequency bands	0.84 = high risk
Cooling energy	Coolant temperature residual under same load/RPM	0.72 = cooling degradation
Injector/combustion energy	Pressure/load residual and RPM instability	0.51 = watch state
Oil/lubrication energy	Temperature + vibration rise pattern	0.38 = early drift
Electrical energy	Current/voltage residual if available	0.18 = normal

3.3 Why Energy Field Helps in Presentation

- The judge does not need to understand all ML internals; red/orange component cells explain where the issue is.
- It separates “model confidence” from “machine stress”, making the dashboard more trustworthy.
- It creates a strong visual differentiator compared to normal predictive-maintenance charts.

4. Residual Digital Twin

The residual digital twin is a lightweight model of healthy behavior. It predicts what the telemetry should look like under the current operating context. The dashboard then plots the difference between actual telemetry and expected telemetry.

4.1 Twin Inputs and Outputs

Input to twin	Output from twin	Residual used for
Past telemetry window	Expected next telemetry or expected feature vector	Anomaly detection and energy field
Operating context: RPM, load, temperature	Healthy baseline under same context	Avoids false alarms during high-load operation
Recent health/RUL history	Future degradation projection	Estimated failure date and what-if analysis

4.2 How to Build the Twin in MVP

1. Start with a simple healthy baseline model using rolling mean + operating-context normalization.
2. Upgrade to an ML twin predictor using RandomForestRegressor, XGBoost, or a small LSTM to predict expected features.

- 3. Calculate residual = actual - expected for each channel and component.
- 4. Use residual trend as one input to the anomaly model and the RUL projection.
- 5. Add a what-if slider in the dashboard: load_factor = 0.8 / 1.0 / 1.2 to shift degradation curve.

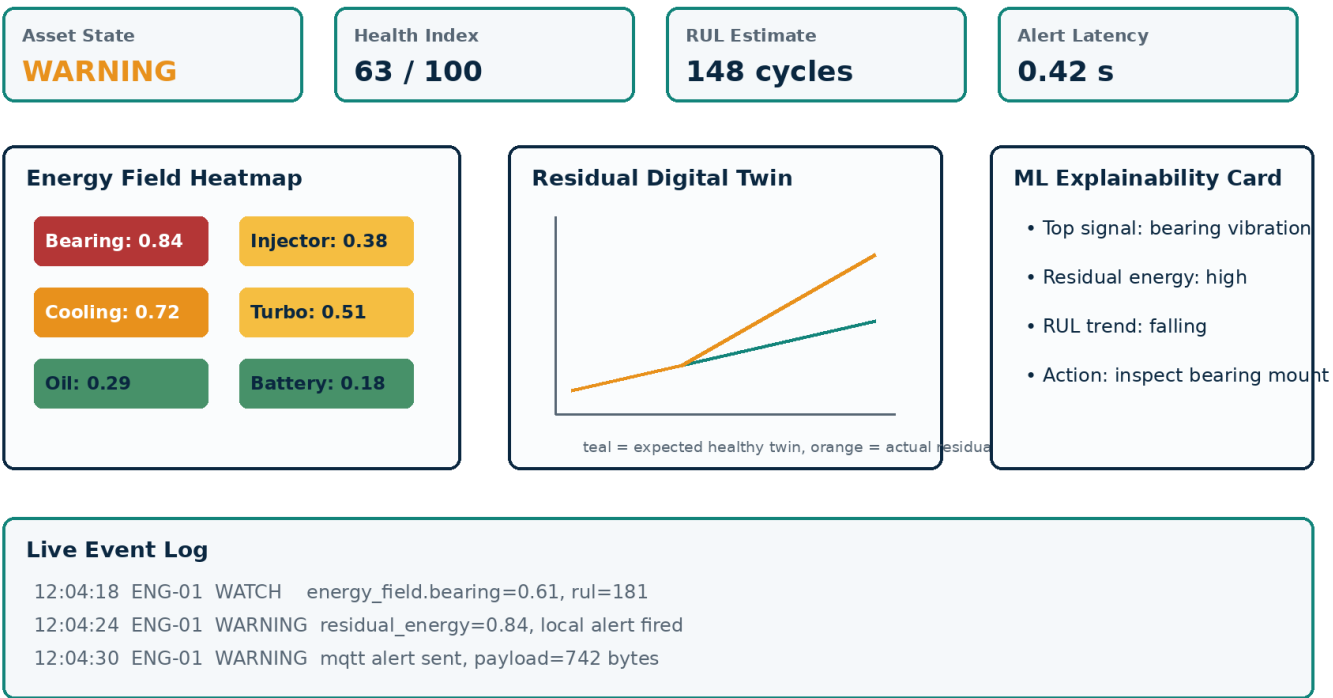
4.3 Residual Twin Card in Dashboard

- Expected line: what the healthy system should do.
- Actual line: current telemetry feature or health score.
- Residual band: acceptable healthy variation.
- Drift zone: when actual leaves the band for multiple windows, state becomes WATCH/WARNING.

5. Telemetry Dashboard Design

The dashboard should be the main visible product. The edge service does the work, but the dashboard tells the story.

Telemetry Dashboard Wireframe



5.1 Dashboard Pages

Page	Widgets	Purpose
Fleet Overview	Asset cards, state colors, last seen, health index	Show all vehicles/engines quickly
Asset Detail	Live telemetry charts, RUL countdown, anomaly score	Deep view for one asset
Energy Field	Component heatmap, energy score timeline	Show where stress is forming
Residual Twin	Expected vs actual plot, residual band, what-if slider	Explain why the system is abnormal
ML Explanation	Top signals, model confidence, fault class, recommended action	Make AI output understandable
Event Log	Last 50 alerts, latency, payload size, sync status	Prove system is live and auditable

5.2 Dashboard States

State	Condition	UI color/action
HEALTHY	Energy low, residual inside band, RUL stable	Green tile, no alert
WATCH	Energy rising or residual drifts for few windows	Yellow tile, technician awareness
WARNING	High energy / anomaly and RUL trend falling	Orange tile, local alert, maintenance suggestion
CRITICAL	Severe energy/residual or RUL below threshold	Red tile, urgent service action

6. Technical Architecture and Repository

This structure keeps the system clean and buildable. The dashboard should not directly run ML; it reads stored events and summaries. The edge app owns inference and decisions.

```
vitals-edge/
├── vitals_common/
│   ├── schemas.py      # SensorWindow, Prediction, Alert, HealthEvent
│   ├── config.py       # .env settings
│   └── preprocess.py    # shared scaling, windowing, FFT, feature creation
├── vitals_data/
│   ├── loaders.py      # C-MAPSS, CWRU, AI4I loaders
│   └── replay.py        # streams data like live telemetry
├── vitals_train/
│   ├── train_twin.py    # healthy baseline / residual twin predictor
│   ├── train_anomaly.py # autoencoder / isolation forest
│   ├── train_rul.py     # CNN-LSTM / regression RUL model
│   └── export.py        # ONNX export and latency benchmark
├── vitals_edge/
│   ├── inference.py     # runs twin + energy field + ML
│   ├── energy_field.py  # component energy scoring
│   ├── decision.py      # health state machine
│   ├── alert.py         # local alert + MQTT buffered publish
│   └── app.py           # FastAPI /health /predict /alerts
├── vitals_cloud/
│   ├── subscriber.py    # MQTT subscriber
│   ├── storage.py       # SQLite/InfluxDB adapter
│   ├── twin.py          # degradation projection / what-if
│   └── dashboard.py     # Streamlit dashboard
└── scripts/demo.py      # one-command healthy -> fault -> alert demo
```

6.1 API Endpoints

Endpoint	Method	Returns
/health	GET	Service status, model versions, latest asset status
/predict	POST	Prediction + health state from a SensorWindow
/alerts	GET	Recent local alerts/events
/energy-field/{asset_id}	GET	Latest component energy map
/twin/{asset_id}	GET	Residual trend and degradation projection

6.2 Storage Schema

Field	Purpose
asset_id, timestamp	Identify asset and time window
state, health_index	Dashboard status and overall health
anomaly_score, rul_estimate	ML outputs used for decision
energy_json, residual_json	Energy-field map and residual twin details
fault_class, message	Human-friendly explanation and action
latency_ms, payload_bytes, synced_flag	Demo proof: speed, compact packet, offline sync

7. ML Model Plan

Model	Input	Output	Why needed
Healthy Twin Predictor	Recent telemetry features + load context	Expected feature vector	Creates residuals and explains drift
Residual Anomaly Model	Residual features + energy field	Anomaly score 0-1	Detects unknown faults without many labels
RUL Regressor	C-MAPSS windows / health trend	Remaining useful life	Shows countdown and maintenance timing
Fault Classifier optional	Energy field + labeled vibration windows	Bearing / imbalance / cooling / injector	Adds human-friendly fault name
Decision Engine	Anomaly + RUL + energy + residual trend	HEALTHY/WATCH/WARNING/CRITICAL	Converts ML into action

7.1 Training Order

1. Build preprocessing and replay first. Do not train until the feature path is stable.
2. Train the healthy twin predictor on healthy/normal windows only.
3. Generate residual features from training data and save residual scaler.
4. Train anomaly model using healthy residuals and seeded/fault data for validation.
5. Train RUL model on C-MAPSS FD001 first. Keep FD002-FD004 as later upgrade.
6. Export model artifacts and thresholds together with version tags.

7.2 Decision Logic

```
if anomaly_score < 0.45 and total_energy < 0.40:
    state = "HEALTHY"
elif anomaly_score < 0.65 or residual_trend_is_rising:
    state = "WATCH"
elif anomaly_score >= 0.65 or energy_component_max >= 0.70 or rul_estimate < warning_rul:
    state = "WARNING"
elif anomaly_score >= 0.85 or energy_component_max >= 0.85 or rul_estimate < critical_rul:
    state = "CRITICAL"

# Use hysteresis: do not jump down immediately; require 3-5 healthy windows.
```

8. Development Runbook - Entire Flow

This section is the practical build order. Follow this exactly so the project does not become too broad.

8.1 Setup Commands

```
# Windows PowerShell or terminal
git clone <your-repo-url> vitals-edge
cd vitals-edge
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate     # Linux/Mac
python -m pip install --upgrade pip
pip install -e .[train,edge,cloud]
copy .env.example .env
```

8.2 Build Milestones

Days	Build	Command / Gate
1-2	Repo, schemas, config, common preprocessing	make test
3	Dataset loaders + replay harness	make replay -- --fault
4-5	Twin predictor + energy field engine	pytest tests/test_energy_field.py
6-7	Anomaly + RUL model training	make train-anomaly && make train-rul
8	Model export + inference runner	make export && make bench-latency
9	FastAPI edge service	make run-edge; POST /predict works
10	MQTT subscriber + SQLite storage	docker compose up -d; alert stored
11	Streamlit dashboard	make run-dashboard
12	End-to-end + offline demo	make demo; make test-e2e

8.3 Run the Demo Flow

```
# Terminal 1: start broker / storage
docker compose up -d

# Terminal 2: start edge service
make run-edge

# Terminal 3: start dashboard
make run-dashboard

# Terminal 4: replay telemetry with fault injection
make replay -- --asset-id ENG-01 --fault --rate 20

# One-command version for final stage
make demo
```

8.4 What the Demo Must Show

1. Dashboard starts with ENG-01 in HEALTHY state.
2. Replay injects a developing fault after 20-30 seconds.
3. Residual twin line starts drifting away from healthy expected line.
4. Energy field cell, for example Bearing, turns yellow/orange/red.
5. ML anomaly score increases and RUL countdown drops.
6. Decision engine changes state to WATCH then WARNING.
7. Local alert fires even if network/broker is stopped.
8. When broker returns, buffered alerts sync into dashboard/event log.

9. Module-Level Build Details

9.1 vitals_common/preprocess.py

- `make_windows(data, window_size, stride)`: creates sliding windows.
- `fit_scaler(windows)`: fits scaler on healthy training data only.
- `apply_scaler(window, scaler)`: applies saved scaler at serving time.
- `fft_features(window)`: extracts RMS, kurtosis, band power, peak frequency.
- `make_features(window)`: one common function used by train and edge.

9.2 vitals_edge/energy_field.py

- Maps signals to components: bearing, cooling, injector/combustion, oil, electrical.
- Computes component energy from residual RMS, FFT band power, temperature gradient and pressure deviation.
- Returns energy map JSON and total health index.
- Keeps formulas simple enough to explain and fast enough for edge use.

9.3 vitals_cloud/dashboard.py

- Uses Streamlit + Plotly for fast development.
- Auto-refresh every 1-2 seconds.
- Reads latest events from SQLite first; MQTT live subscription can be added after MVP.
- Shows last seen timestamp so dashboard clearly handles connection loss.

10. KPIs and Acceptance Gates

KPI	MVP target	How to prove
Edge-to-alert latency	p95 < 2 seconds, inference target < 300 ms	test_e2e latency report
Alert payload size	< 1 KB	schema test serializes realistic payload
Offline resilience	Local alert works with broker down	test_offline.py
Anomaly quality	Clear separation healthy vs fault	ROC-AUC / confusion matrix
RUL usefulness	Stable countdown trend; target RMSE 15-20 cycles first	metrics_report.md
Dashboard clarity	Fault visible within seconds	demo checklist + screen recording

10.1 Final Demo Storyboard

Time	Action	What judges see
0:00-0:20	Open dashboard and start healthy replay	Green health tile, low energy field
0:20-0:40	Inject fault	Residual line drifts, bearing energy rises
0:40-1:00	Decision changes state	WATCH/WARNING, RUL decreases
1:00-1:20	Cut broker/network	Dashboard last-seen freezes but local alert continues
1:20-1:40	Reconnect broker	Buffered alerts sync into event log
1:40-2:00	Explain maintenance card	Root cause, sensor reason, recommended action

11. Risks and Fixes

Risk	Fix
Energy field sounds too theoretical	Define it clearly as telemetry residual-energy visualization, not a physics claim.
No physical engine access	Use dataset replay as live telemetry and keep real sensors as

	optional roadmap.
Model accuracy not perfect	Use demo as decision support; show trends, not absolute promises.
Dashboard becomes too heavy	Start with Streamlit + SQLite; move to Grafana/InfluxDB only if time allows.
False alarms	Use residual twin + operating-context normalization + hysteresis.
Stage demo failure	Keep deterministic replay and a pre-recorded backup video.

12. Simple Explanation Script

“Our project is VITALS-Edge, a telemetry-based health dashboard for engines and rotating machinery. Instead of sending raw sensor data to the cloud, we process it on the edge device. The system builds a healthy residual digital twin, compares actual telemetry with expected behavior, converts the deviation into an energy-field heatmap, and then uses ML to detect anomaly and estimate remaining useful life. The dashboard shows where the fault is forming, why the model detected it, and what action the technician should take. The key demo is that even if the network is disconnected, the edge system continues detecting locally and buffers alerts for later sync.”

12.1 One-Slide Architecture Sentence

Telemetry comes in, the residual twin predicts healthy behavior, the energy field visualizes abnormal stress, ML converts it into anomaly/RUL, and the dashboard turns it into maintenance action.

Appendix A - Ready-to-Paste Master Prompt for Claude/Cursor

```
ROLE: You are a senior ML + backend engineer helping build VITALS-Edge: a telemetry dashboard for engine health.
GOAL: Build an edge-first system that turns live/dataset telemetry into residual digital twin drift, energy-field heatmap, anomaly score, RUL estimate, and maintenance alerts.
STACK: Python 3.11, FastAPI, Streamlit, SQLite/InfluxDB, MQTT, PyTorch/scikit-learn, ONNX Runtime.
RULES: Keep inference on edge, keep cloud thin, use one shared preprocessing module, make the demo work without real hardware by dataset replay, and keep alert JSON under 1 KB.
FIRST TASK: Create repo scaffold with vitals_common, vitals_data, vitals_train, vitals_edge, vitals_cloud, tests, scripts, docs. Include Makefile targets install, test, replay, run-edge, run-dashboard, demo.
```